

Laporan Tugas Kecil 3 IF2211 Strategi Algoritma
Ice Sliding Puzzle Solver
Semester II Tahun Ajaran 2025/2026



Disusun oleh:

Dika Pramudya Nugraha (13524132)
Daniel Putra Rywandi S (13524143)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026



Kata Pengantar

Dokumen Laporan Tugas Kecil 3 ini disusun mahasiswa Teknik Informatika angkatan 2024. Adapun susunan anggota kelompok kami secara detail sebagai berikut:

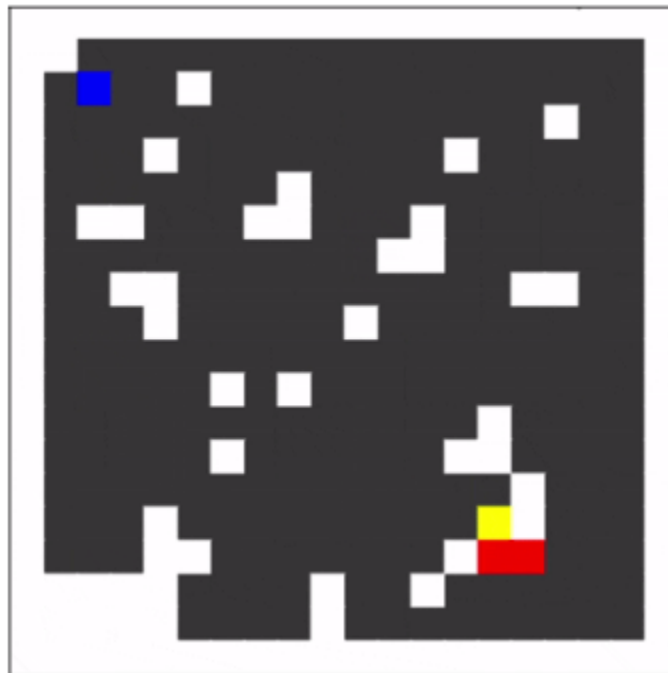
1. **Dika** Pramudya Nugraha (NIM: 13524**132**)
2. **Daniel** Putra Rywandi S (NIM: 13524**143**)

Daftar Isi

Kata Pengantar.....	1
Daftar Isi.....	2
BAB 1.....	3
DESKRIPSI ALGORITMA PATHFINDING.....	3
1.1 Algoritma Uniform Cost Search (UCS).....	4
1.2 Algoritma Greedy Best-First Search (GBFS).....	5
1.3 Algoritma A (A-Star) Search*.....	6
1.4 Breadth-First Search (BFS).....	7
1.5 Iterative Deepening A-Star (IDA*).....	8
BAB 2.....	9
ANALISIS TEORITIS ALGORITMA.....	9
2.1 Definisi Fungsi Evaluasi $f(n)$ dan $g(n)$	9
2.2 Analisis Admisibilitas Heuristik A*.....	9
2.2 Perbandingan Teoritis Algoritma.....	10
BAB 3.....	12
IMPLEMENTASI PROGRAM.....	12
3.1 Repository Program.....	12
3.2 Struktur Source Code.....	12
3.2.1 Modul Types (Types.hpp).....	12
3.2.2 Modul Map (Map.hpp dan Map.cpp).....	12
3.2.3 Modul Solver (Solver.hpp dan Solver.cpp).....	15
3.2.4 Modul Main (main.cpp).....	16
3.2.5 GUI (MainWindow.hpp dan MainWindow.cpp).....	21
3.2.6 Modul Render Papan (BoardWidget.hpp dan BoardWidget.cpp).....	22
BAB 4.....	25
PENGUJIAN DAN ANALISIS HASIL.....	25
5.1 Test Case 1.....	25
5.2 Test Case 2.....	30
5.3 Test Case 3.....	38
5.4 Test Case 4.....	43
5.5 Test Case 5.....	43
5.6 Test Case 6.....	44
LAMPIRAN.....	60
PERNYATAAN TIDAK MELAKUKAN KECURANGAN.....	61
DAFTAR PUSTAKA.....	62

BAB 1

DESKRIPSI ALGORITMA PATHFINDING

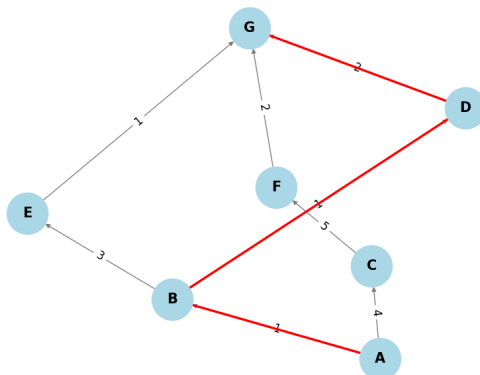


Gambar 1. Visualisasi Permainan Ice Sliding Puzzle Solver

Permainan *Ice Sliding Puzzle* menghadirkan tantangan pencarian rute (pathfinding) yang unik dibandingkan dengan penelusuran grid konvensional. Pada persoalan ini, aktor (direpresentasikan dengan huruf 'Z') berada di atas permukaan es yang licin, sehingga setiap inisiasi pergerakan (Atas, Bawah, Kiri, Kanan) akan mengakibatkan aktor meluncur terus-menerus tanpa bisa berhenti di petak sembarang. Aktor hanya akan berhenti bergerak apabila menabrak rintangan atau batu (direpresentasikan dengan 'X'). Selain itu, aktor harus menghindari lava ('L') yang menyebabkan kegagalan (game over), menghindari meluncur ke luar batas peta tanpa rintangan, dan diwajibkan untuk mengumpulkan angka yang tersebar di peta (0 hingga maksimal 9) secara berurutan sebelum mencapai titik tujuan akhir ('O').

Untuk menyelesaikan persoalan dengan kompleksitas state ruang pencarian (search space) tersebut, program ini mengimplementasikan tiga algoritma pencarian utama dan satu algoritma tambahan.

1.1 Algoritma Uniform Cost Search (UCS)



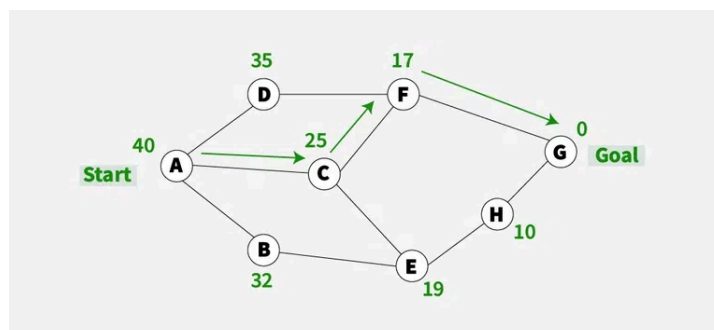
Gambar 2. Visualisasi Algoritma UCS

Uniform Cost Search (UCS) adalah algoritma pencarian jalur yang memperluas simpul dengan *cost* (biaya) terendah terlebih dahulu, memastikan bahwa jalur ke simpul tujuan memiliki biaya minimum. Tidak seperti algoritma pencarian lainnya seperti *Breadth-First Search (BFS)*, UCS memperhitungkan biaya setiap jalur, sehingga cocok untuk graf berbobot di mana setiap sisi memiliki biaya yang berbeda.

Dalam penyelesaian *Ice Sliding Puzzle* ini, setiap kali algoritma mengekstraksi sebuah simpul kandidat dari *priority queue* yang dipastikan memiliki biaya kumulatif terkecil pada iterasi tersebut. Jika aktor telah mencapai petak tujuan ('O') dan mengumpulkan seluruh urutan angka, pencarian selesai. Apabila kondisi kemenangan belum tercapai, UCS akan mengekskspansi simpul tersebut dengan membangkitkan semua kemungkinan cabang suksesor yang merepresentasikan pergerakan fisik aktor ke empat arah mata angin, di mana setiap inisiasi arah akan memicu mekanisme meluncur tanpa henti di atas permukaan es, dan pergerakan ini baru akan terhenti tepat satu petak sebelum karakter menabrak rintangan batu yang disimbolkan dengan huruf 'X'.

Selama proses simulasi luncuran yang masif ini berlangsung, UCS akan mengevaluasi validitas lintasan secara ketat, apabila rute tersebut menuntun aktor menyentuh petak lava ('L') yang menyebabkan *game over* atau tergelincir melewati batas tepi ukuran papan tanpa adanya dinding penahan, maka cabang pergerakan tersebut akan dideklarasikan sebagai langkah tidak valid dan secara otomatis dipangkas (*pruned*) dari ruang pencarian. Sebaliknya, untuk pergerakan yang sah, penambahan biaya $g(n)$ dihitung dari total bobot petak yang dilewati selama meluncur (termasuk petak berhenti, namun tidak termasuk petak awal). *State* baru berupa kombinasi koordinat dan progres angka ini kemudian dicek dengan daftar *visited states*. Suksesor hanya akan dimasukkan kembali ke *Priority Queue* jika *state* tersebut belum pernah dikunjungi atau menawarkan *cost* yang lebih murah, sehingga menjamin ditemukannya solusi yang paling optimal.

1.2 Algoritma Greedy Best-First Search (GBFS)



Gambar 3. Visualisasi Algoritma GBFS

Greedy Best-First Search adalah algoritma pencarian yang mencoba menemukan jalur paling menjanjikan dari titik awal yang diberikan ke suatu tujuan. Algoritma ini memprioritaskan jalur yang tampak paling menjanjikan, terlepas dari apakah jalur tersebut benar-benar jalur terpendek atau tidak. Algoritma ini bekerja dengan mengevaluasi estimasi sisa jarak dari setiap kemungkinan jalur menuju target yang secara matematis direpresentasikan sebagai fungsi heuristik $h(n)$ dan kemudian memperpanjang jalur dengan nilai tebakan terendah. Proses ekspansi yang secara total mengabaikan akumulasi biaya aktual ($g(n)$) yang telah dihabiskan ini terus diulang hingga tujuan tercapai. Karakteristik ini membuat GBFS memiliki kecepatan komputasi yang luar biasa dalam menembus ruang pencarian, meskipun konsekuensinya adalah hilangnya jaminan bahwa solusi lintasan yang dihasilkan merupakan rute yang optimal.

Dalam implementasinya pada penyelesaian *Ice Sliding Puzzle*, GBFS memanfaatkan struktur data *priority queue* yang secara diurutkan berdasarkan nilai heuristik paling minimum. Setiap kali algoritma melakukan ekstraksi simpul, prioritas utama diberikan kepada state yang secara kalkulasi spasial dinilai paling dekat dengan target angka yang sedang dicari atau menuju titik akhir 'O'. Ketika aktor meluncur melintasi es dan terhenti di depan petak rintangan batu ('X'), algoritma tidak akan menjumlahkan seberapa besar cost traversal dari petak-petak yang baru saja dilewati, melainkan langsung mengevaluasi ulang estimasi sisa jarak dari titik perhentian tersebut ke target sasaran berikutnya.

Proses ekspansi ini terus diulang dengan membangkitkan pergerakan arah yang valid, sembari mengeliminasi cabang lintasan yang berujung pada petak lava ('L') atau meluncur bebas melewati batas papan. Karena pengambilan keputusannya sangat bergantung pada kedekatan geometris secara heuristik semata, GBFS memiliki kelemahan struktural di mana aktor dapat dengan mudah terjebak ke dalam lintasan yang membebankan biaya ubin sangat tinggi.

1.3 Algoritma A (A-Star) Search*

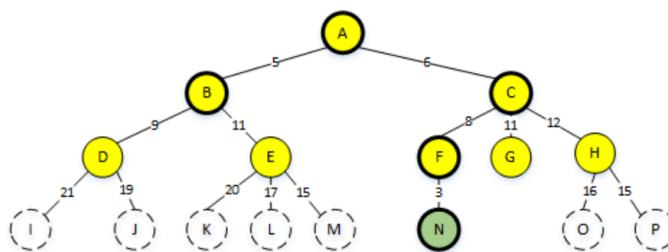
Algoritma *A-Star* merupakan pengembangan dari algoritma BFS (*Breadth First Search*) sehingga dasar pencarian dan algoritma hampir sama. Perbedaannya yaitu algoritma *A-Star* akan memilih jalur dengan nominal nilai atau biaya terkecil. Algoritma ini pertama kali dideskripsikan pada tahun 1968 oleh Peter Hart, Nils Nilsson, dan Bertram Raphael, dengan rumus:

$$f(v) = g(s \rightarrow v) + h(v \rightarrow r)$$

Persamaan digunakan untuk mengisi nilai masing-masing node tetangga selama proses pencarian jalur terjadi, dimana:

- $g(s \rightarrow v)$: Estimasi biaya/jarak dari titik start (s) ke titik v .
- $h(v \rightarrow r)$: Estimasi biaya/jarak tertentu (unik) dari titik v ke titik goal (r).
- $f(v)$: Total biaya/jarak yang berasal dari titik awal ke titik goal yang melewati titik v

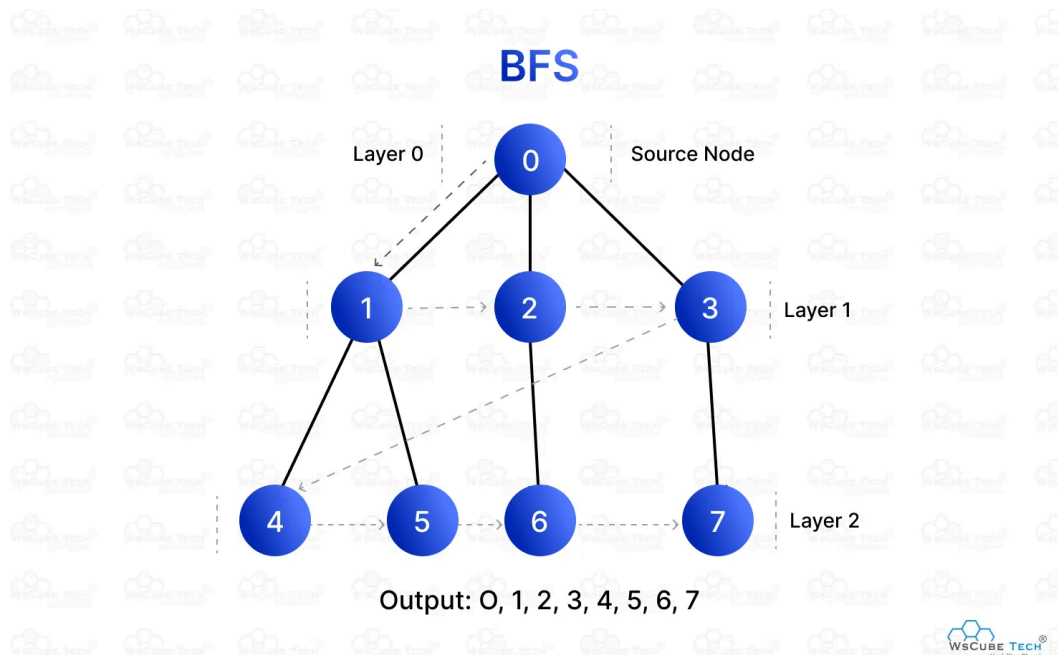
Algoritma *A-Star* membutuhkan variabel khusus yaitu variabel open agar tidak adanya pemeriksaan dua kali. Lalu algoritma *A-Star* akan mendaftarkan node tetangga dari setiap node yang saat ini sedang dikunjungi. Tetapi pada perulangan selanjutnya algoritma *A-Star* akan memilih nilai terendah dari kumpulan node pada variabel open yang terdaftar. Sehingga properti nilai pada masing-masing node adalah mutlak ada. Berikut adalah graf pohon dari algoritma *A-Star*:



Gambar 4. Visualisasi Algoritma *A-Star*

Dimana node A mencari node N, urutan pemeriksaan graf pohon diatas adalah A, B, C, F, dan terakhir N, yaitu node goal ditemukan. Pada graf tersebut nilai masing-masing node dihitung dengan Persamaan 1. Garis putus-putus pada graf tersebut menandakan area yang belum dikunjungi, garis tegas yang tidak tebal adalah node yang akan dikunjungi selanjutnya (masuk ke dalam variabel open), sedangkan garis tebal adalah node yang telah dikunjungi dan diberi tanda closed agar tidak dikunjungi kembali.

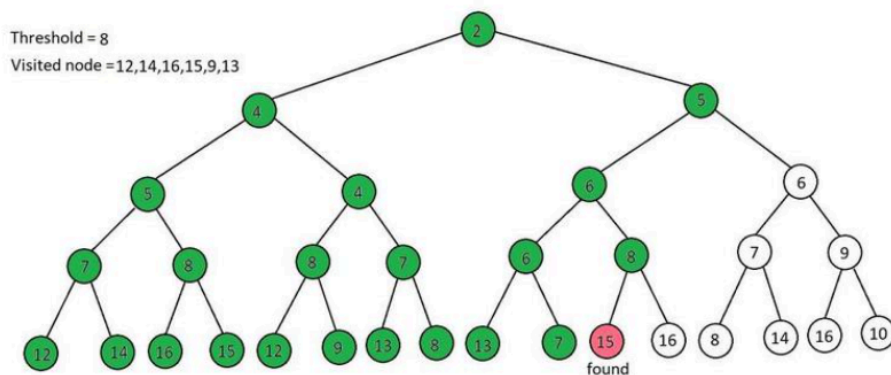
1.4 Breadth-First Search (BFS)



Gambar 5. Visualisasi Algoritma BFS

Dalam konteks penelusuran pohon DOM pada tugas ini, salah satu strategi algoritma yang digunakan adalah *Breadth-First Search (BFS)* yang melakukan eksplorasi simpul secara melebar. Algoritma ini bekerja dengan mengunjungi simpul root terlebih dahulu, kemudian secara sistematis menjelajahi seluruh simpul anak pada tingkat kedalaman k sebelum berpindah ke tingkat kedalaman $k + 1$. BFS sangat bergantung pada struktur data antrian (*Queue*) yang mengikuti prinsip *First-In-First-Out* (FIFO) untuk mengelola urutan simpul yang akan dikunjungi. Secara analitis, kompleksitas waktu dari algoritma ini adalah $O(V + E)$ di mana V adalah jumlah simpul dan E adalah jumlah busur, karena setiap elemen dalam pohon DOM akan diperiksa setidaknya satu kali untuk mencocokkan kriterianya dengan *CSS Selector* yang diberikan. Keunggulan utama BFS dalam penelusuran DOM adalah kemampuannya untuk menemukan elemen yang terletak paling dekat dengan *root* dalam waktu yang relatif cepat serta kemudahan dalam memvisualisasikan jalur penelusuran per level kedalaman.

1.5 Iterative Deepening *A-Star* (IDA*)



Gambar 5. Visualisasi Algoritma IDA-Star

Algoritma *Iterative Deepening A-Star* (IDA*) merupakan algoritma tingkat lanjut dari gabungan algoritma *Depth First Search* (DFS) dan *A-Star* (A*) yang dirancang secara spesifik untuk mengatasi kelemahan konsumsi memori eksponensial pada *priority queue* dengan memadukan keandalan heuristik $f(n)$ dan efisiensi ruang dari pencarian mendalam (DFS). Dalam penerapannya pada penyelesaian *Ice Sliding Puzzle*, algoritma ini tidak mengekskansi simpul secara radian dan serentak, melainkan batas pemotongan. Pada awal iterasi, nilai *threshold* diinisialisasi berdasarkan nilai fungsi evaluasi $f(n)$ dari simpul awal pergerakan karakter. Selama proses penelusuran pada setiap iterasi, program akan membangkitkan cabang pergerakan suksesor yang memicu aktor meluncur melintasi es hingga terhenti secara mekanis di depan petak rintangan batu ('X'), sambil memvalidasi dan secara otomatis memangkas lintasan tidak sah yang menyentuh petak lava ('L') atau meluncur keluar dari batas dimensi papan permainan. Apabila pergerakan luncuran tersebut valid dan nilai $f(n)$ yang baru masih berada di bawah atau sama dengan batas *threshold*, penelusuran DFS akan terus menukik tajam mengeksplorasi cabang tersebut hingga aktor berhasil mendarat di petak tujuan akhir 'O', dengan prasyarat mutlak bahwa seluruh sekuens angka telah tuntas dikumpulkan secara berurutan.

Namun, untuk setiap titik henti luncuran ketika hasil kalkulasi nilai evaluasi total $f(n)$ ternyata melampaui batas *threshold* yang sedang berlaku, ekspansi pada cabang tersebut akan langsung dihentikan secara prematur. Nilai $f(n)$ yang berlebih tersebut kemudian akan direkam sebagai kandidat batas baru untuk iterasi selanjutnya. Apabila satu siklus penelusuran DFS berakhir tanpa menemukan solusi absolut, algoritma akan memperbarui nilai *threshold* menggunakan nilai $f(n)$ terkecil dari sekumpulan simpul yang sebelumnya terpotong pada iterasi yang lalu, dan kemudian mengulangi seluruh proses penelusuran dari titik awal kembali.

BAB 2

ANALISIS TEORITIS ALGORITMA

2.1 Definisi Fungsi Evaluasi $f(n)$ dan $g(n)$

Dalam konteks penyelesaian permasalahan *Ice Sliding Puzzle*, pencarian lintasan yang optimal sangat bergantung pada formulasi fungsi evaluasi yang digunakan oleh algoritma. Fungsi $g(n)$ merepresentasikan *actual cost* yang telah diakumulasikan dari simpul awal hingga mencapai simpul n yang sedang dievaluasi. Berdasarkan spesifikasi aturan permainan, biaya satu pergerakan *sliding* tidak dihitung sebagai satu langkah bernilai mutlak, melainkan merupakan hasil penjumlahan dari *cost* setiap petak ubin yang dilewati oleh karakter di atas permukaan es. Kalkulasi biaya ini secara spesifik mengecualikan petak awal tempat karakter mulai meluncur, namun tetap memperhitungkan petak akhir tempat karakter tersebut berhenti tegak karena rintangan. Dengan demikian, $g(n)$ secara presisi merekam seberapa mahal lintasan nyata yang telah ditempuh sejauh ini tanpa adanya unsur tebakan.

Di sisi lain, fungsi $f(n)$ bertindak sebagai fungsi evaluasi keseluruhan yang menjadi landasan utama pengambilan keputusan, khususnya pada algoritma A^* . Fungsi $f(n)$ didefinisikan sebagai estimasi total biaya lintasan dari titik awal, melewati simpul n , dan berlanjut hingga mencapai kondisi tujuan akhir. Formulasi matematis dari fungsi ini adalah $f(n) = g(n) + h(n)$, di mana $h(n)$ merupakan nilai heuristik atau tebakan sisa biaya dari simpul n menuju tujuan. Dengan menggabungkan pengeluaran nyata di masa lalu ($g(n)$) dan estimasi biaya di masa depan ($h(n)$), fungsi evaluasi ini memungkinkan algoritma untuk memprioritaskan ekspansi secara cerdas pada rute-rute yang secara matematis menawarkan total pengeluaran paling efisien.

2.2 Analisis Admisibilitas Heuristik A^*

Syarat mutlak algoritma A^* dapat menjamin penemuan lintasan yang optimal adalah penggunaan fungsi heuristik $h(n)$ yang bersifat *admissible* (dapat diterima). Sebuah heuristik dikategorikan *admissible* apabila nilai estimasinya tidak pernah melampaui biaya terendah untuk mencapai tujuan dari simpul tersebut, yang secara formal dinotasikan dengan $h(n) \leq h^*(n)$, $h^*(n)$ adalah biaya paling minimum. Dalam permainan *Ice Sliding Puzzle*, karakter hanya dapat bergerak secara vertikal dan horizontal, sehingga metrik jarak geometris yang paling logis adalah *Manhattan Distance*. Namun, karena setiap ubin di papan memiliki nilai *cost* yang bervariasi dari yang termurah hingga sangat mahal, penggunaan *Manhattan Distance* hanya merepresentasikan jarak spasial, bukan biaya.

Untuk memastikan agar heuristik tetap *admissible*, perhitungan jarak geometris tersebut harus dimodifikasi menjadi estimasi biaya terendah. Modifikasi yang diimplementasikan adalah dengan mengalikan *Manhattan Distance* (dari posisi karakter ke target sasaran berikutnya) dengan nilai *cost* paling minimum yang ada pada konfigurasi papan permainan tersebut secara keseluruhan. Skenario teoretis paling ideal yang mungkin

terjadi adalah jika aktor dapat meluncur langsung ke arah target di mana seluruh petak lintasan yang dilewatinya merupakan petak termurah di peta.

Dalam realitas permainan, karakter pasti akan berhadapan dengan rintangan batu ('X') yang mengharuskan manuver memutar, atau terpaksa melintasi petak-petak dengan *cost* tinggi. Hal ini mengonfirmasi bahwa biaya lintasan aktual $h^*(n)$ pada akhirnya akan selalu lebih besar atau setidaknya sama dengan estimasi heuristik modifikasi kita. Konsekuensinya, fungsi heuristik ini tidak akan pernah melakukan *overestimate* terhadap biaya sisa, terbukti *admissible*, dan secara matematis akan selalu menuntun algoritma A* pada lintasan solusi yang paling optimal.

2.2 Perbandingan Teoritis Algoritma

Dalam penyelesaian ruang pencarian yang ekstensif pada *Ice Sliding Puzzle*, algoritma *Uniform Cost Search* (UCS), *Greedy Best-First Search* (GBFS), A*, *Breadth-First Search* (BFS), dan *Iterative Deepening A** (IDA*) menunjukkan karakteristik dan *trade-off* performa yang sangat bertolak belakang. Algoritma UCS memiliki keunggulan absolut dalam memberikan jaminan penemuan solusi paling optimal dan pasti menemukan jalan keluar bila eksis. Karena UCS beroperasi secara buta hanya berpatokan pada bobot aktual $g(n)$, algoritma ini terpaksa mengekskspansi simpul pencarian secara radial ke segala arah. Implikasinya, kompleksitas peninjauan iterasi menjadi sangat tinggi, memakan alokasi memori antrean yang masif, dan durasi eksekusinya relatif lebih lambat.

Sebagai *baseline* pencarian tak berinformasi (*uninformed search*), algoritma BFS beroperasi dengan mengekskspansi ruang pencarian tingkat demi tingkat (kedalaman secara merata) memanfaatkan struktur data antrean *First-In-First-Out* (FIFO). Dalam konteks permainan ini, BFS secara inheren dijamin komplit dan pasti menemukan solusi dengan jumlah interaksi langkah luncuran paling sedikit. Akan tetapi, karena BFS sama sekali mengabaikan akumulasi bobot atau biaya lintasan, rute yang dihasilkannya hampir dipastikan tidak optimal secara *cost* keseluruhan. Lebih lanjut, ekspansi merata pada setiap tingkat kedalaman memaksa BFS untuk menyimpan seluruh simpul pada level tersebut di dalam memori, memicu kompleksitas ruang eksponensial $O(b^d)$ yang sangat boros dan rawan memicu kehabisan alokasi memori (*Out of Memory*).

Sebaliknya, algoritma GBFS menawarkan kecepatan komputasi yang jauh lebih superior. Dengan hanya mengevaluasi fungsi tebakan heuristik $h(n)$, GBFS secara agresif memprioritaskan lintasan yang secara geometris terasa paling dekat dengan target sasaran tanpa menghiraukan seberapa mahal ongkos langkah yang telah dihabiskan. Walaupun eksekusinya sangat singkat, algoritma ini secara inheren cacat dalam hal optimalitas biaya. GBFS sangat rentan tertipu oleh formasi rintangan yang memaksa karakter melintasi

deretan ubin berbiaya mahal, sehingga solusi yang ditawarkan seringkali memiliki total akumulasi biaya yang membengkak drastis.

Pada algoritma A^* memanifestasikan keunggulan secara teoritis. Dengan mengintegrasikan biaya riil $g(n)$ dari UCS dan panduan heuristik $h(n)$ dari GBFS, A^* mampu memandu pencarian secara langsung menuju target untuk mengeliminasi ribuan iterasi simpul yang tidak relevan, sekaligus tetap membatasi ekspansi pada rute-rute yang mahal. Jika fungsi heuristik yang digunakan terbukti *admissible*, komparasi ini menempatkan A^* sebagai solusi yang paling seimbang, yaitu sangat efisien dalam eksekusi waktu, tetapi tetap mutlak dalam mempertahankan optimalitas biaya pergerakan karakter. Sebagai tambahan, algoritma *Iterative Deepening A^** (IDA^*) hadir untuk menyempurnakan A^* dengan mengeliminasi masalah ledakan memori antrean prioritas melalui pendekatan pencarian DFS berulang yang dibatasi *threshold*, menjadikan memori yang digunakan jauh lebih konstan.

Aspek Komunikasi	UCS	GBFS	A^* Search	IDA^*	BFS
Optimalitas Biaya	Selalu optimal	Tidak Optimal	Selalu Optimal	Selalu Optimal	Tidak optimal (hanya optimal dalam jumlah <i>step</i>)
Kelengkapan	Komplit	Komplit	Komplit	Komplit	Komplit
Efisiensi Waktu Eksekusi	Rendah (paling banyak meninjau iterasi node)	Tinggi (sangat cepat menemukan target)	Sedang - tinggi (bergantung kualitas heuristik)	Sedang (lebih lambat dari A^* karena DFS berulang)	Sangat rendah (pencarian menyebar rata)
Efisiensi Memori	Rendah (Memori eksponensial $O(b^{C^*/\epsilon})$)	Sedang - tinggi	Rendah (memori eksponensial menyimpan Priority Queue)	Sangat Tinggi	Sangata rendah (memori eksponensial $O(b^d)$)
Fungsi Prioritas	$f(n) = g(n)$	$f(n) = h(n)$	$f(n) = g(n) + h(n)$	$f(n) = g(n) + h(n)$ dengan batas <i>cutoff</i> iteratif	FIFO (tanpa perhitungan evaluasi biaya)

BAB 3

IMPLEMENTASI PROGRAM

3.1 Repository Program

Berikut adalah pranala ke *repository* program:

https://github.com/ItsMeD4N/tucil3_13524132_13524143

3.2 Struktur Source Code

Program Ice Sliding Puzzle Solver dikembangkan menggunakan bahasa pemrograman C++ dengan struktur kode yang modular. Pemisahan antara file header (.hpp) dan file implementasi (.cpp) dilakukan untuk menjaga kerapian kode serta mempermudah proses pengembangan maupun debugging. Secara umum, source code program dibagi ke dalam empat modul utama.

3.2.1 Modul Types (Types.hpp)

```
struct State {
    int r, c;
    int mask;
    bool operator==(const State& o) const {
        return r == o.r && c == o.c && mask == o.mask;
    }
};
```

Modul ini menjadi dasar dari seluruh struktur data yang digunakan dalam program. Di dalamnya terdapat beberapa struct penting yang mendukung proses simulasi dan pencarian solusi.

- Point digunakan untuk menyimpan posisi koordinat baris (r) dan kolom (c) pada papan permainan.
- State merepresentasikan kondisi permainan saat tertentu, mencakup posisi karakter serta nilai mask yang digunakan untuk melacak waypoint atau target angka yang sudah dikunjungi.
- Node berfungsi sebagai simpul pada algoritma pencarian (pathfinding), yang menyimpan State, nilai biaya g, nilai heuristik h, serta riwayat pergerakan dalam bentuk string.
- SlideResult digunakan untuk menyimpan hasil simulasi gerakan meluncur, meliputi state akhir, total tambahan biaya lintasan, dan status validitas jalur, misalnya ketika karakter keluar batas atau mengenai lava.

3.2.2 Modul Map (Map.hpp dan Map.cpp)

```
bool Map::load(const string& filename) {
    last_error.clear();
    ifstream file(filename);
    if (!file) {
        last_error = "File tidak dapat dibuka.";
        return false;
    }

    if (!(file >> rows >> cols) || rows <= 0 || cols <= 0) {
        last_error = "Ukuran board tidak valid.";
        return false;
    }

    grid.resize(rows);
    for (int r = 0; r < rows; ++r) {
        if (!(file >> grid[r]) || static_cast<int>(grid[r].size())
!= cols) {
            last_error = "Baris board tidak sesuai ukuran kolom.";
            return false;
        }
    }

    costs.resize(rows, vector<int>(cols));
    for (int r = 0; r < rows; ++r) {
        for (int c = 0; c < cols; ++c) {
            if (!(file >> costs[r][c]) || costs[r][c] < 0) {
                last_error = "Data cost tidak lengkap atau
bernilai negatif.";
                return false;
            }
        }
    }

    vector<Point> temp_wp(10, Point{-1, -1});
    int max_wp = -1;
    int start_count = 0;
    int goal_count = 0;

    for (int r = 0; r < rows; ++r) {
        for (int c = 0; c < cols; ++c) {
            char ch = grid[r][c];
            bool is_digit = (ch >= '0' && ch <= '9');
            bool is_allowed = (ch == '*' || ch == 'X' || ch == 'L'
|| ch == 'O' || ch == 'Z' || is_digit);
            if (!is_allowed) {
                last_error = "Terdapat karakter tile tidak
valid.";
            }
        }
    }
}
```

```
        return false;
    }

    if (ch == 'Z') {
        start_count++;
        start = {r, c};
        grid[r][c] = '*';
    } else if (ch == 'O') {
        goal_count++;
        goal = {r, c};
    } else if (is_digit) {
        int val = ch - '0';
        if (temp_wp[val].r != -1) {
            last_error = "Waypoint angka duplikat.";
            return false;
        }
        temp_wp[val] = {r, c};
        if (val > max_wp) max_wp = val;
    }
}

if (start_count != 1 || goal_count != 1) {
    last_error = "Board harus memiliki tepat satu titik start (Z) dan satu goal (O).";
    return false;
}

total_waypoints = max_wp + 1;
waypoints.clear();
for (int i = 0; i < total_waypoints; ++i) {
    if (temp_wp[i].r == -1) {
        last_error = "Waypoint angka harus berurutan dari 0 tanpa ada yang hilang.";
        return false;
    }
    waypoints.push_back(temp_wp[i]);
}

return true;
}
```

Modul ini bertanggung jawab dalam pengelolaan representasi papan permainan beserta proses pembacaan data peta.

- Parsing File dilakukan melalui fungsi `load(const string& filename)` yang membaca file input `.txt`, menentukan ukuran papan, mengekstrak elemen-elemen peta, serta memetakan nilai cost pada setiap ubin.

- Validasi Peta dilakukan secara otomatis untuk memastikan keberadaan titik awal ('Z') dan tujuan ('O'), sekaligus memeriksa apakah urutan target angka (0–9) tersedia dengan benar.
- Visualisasi disediakan melalui fungsi `printMap`, yang digunakan untuk menampilkan kondisi papan pada Command Line Interface (CLI). Fungsi ini juga menyesuaikan tampilan waypoint yang sudah dilewati menjadi petak kosong (*) dan memperbarui posisi karakter sesuai state saat ini.

3.2.3 Modul Solver (Solver.hpp dan Solver.cpp)

```
SlideResult performSlide(const Map& map, State current, char dir)
{
    int dr = 0, dc = 0;
    if (dir == 'U') dr = -1;
    if (dir == 'D') dr = 1;
    if (dir == 'L') dc = -1;
    if (dir == 'R') dc = 1;

    int r = current.r;
    int c = current.c;
    int mask = current.mask;
    int cost = 0;
    bool moved = false;

    while (true) {
        int nr = r + dr;
        int nc = c + dc;

        if (nr < 0 || nr >= map.rows || nc < 0 || nc >=
map.grid[nr].size()) {
            return {State{r, c, mask}, 0, false};
        }

        char cell = map.grid[nr][nc];

        if (cell == 'X') {
            break;
        }

        moved = true;
        cost += map.costs[nr][nc];
        r = nr;
        c = nc;

        if (cell == 'L') {
            return {State{r, c, mask}, 0, false};
        }
    }
}
```



```
        if (cell >= '0' && cell <= '9') {
            int wp = cell - '0';
            if (wp > mask) {
                return {State{r, c, mask}, 0, false};
            } else if (wp == mask) {
                mask++;
            }
        }
    }

    if (!moved) return {State{r, c, mask}, 0, false};

    return {State{r, c, mask}, cost, true};
}
```

Modul ini merupakan inti logika program yang menangani simulasi permainan sekaligus implementasi algoritma pencarian solusi.

- Mekanika Slide diimplementasikan melalui fungsi `performSlide`. Fungsi ini mensimulasikan pergerakan karakter yang terus meluncur ke arah tertentu (U, D, L, R) hingga berhenti karena menabrak batu ('X'). Selama proses tersebut, fungsi juga menghitung total cost lintasan dan langsung menghentikan simulasi apabila karakter keluar batas atau mengenai lava ('L').
- Heuristik digunakan pada algoritma IDA melalui fungsi `computeHeuristic`. Fungsi ini menyediakan tiga pendekatan perhitungan, yaitu Manhattan Distance (H1), Euclidean Distance (H2), dan perhitungan jumlah waypoint yang belum dikunjungi (H3).
- Algoritma mencakup implementasi berbagai algoritma pencarian. Fungsi `runSearch` digunakan untuk algoritma berbasis priority queue seperti UCS dan GBFS. Sementara itu, algoritma IDA diimplementasikan melalui `runIDAStar` dan `idaDfs`, sedangkan BFS dijalankan menggunakan `runBFS`.
- Dispatcher direalisasikan melalui fungsi `solve`, yang berperan sebagai penghubung antara input pengguna dengan algoritma yang dipilih. Fungsi ini menerima pilihan algoritma dan heuristik, kemudian menjalankan metode pencarian yang sesuai.

3.2.4 Modul Main (main.cpp)

```
int main() {
    string filename;
    cout << ">> Masukan file input :";
    cin >> ws;
    getline(cin, filename);
}
```

```
Map map;
if (!map.load(filename)) {
    cout << "Gagal membaca file. " << map.last_error << "\n";
    return 1;
}

string algo;
while (true) {
    cout << ">> Algoritma apa yang anda pilih?
(UCS/BFS/GBFS/A*/IDA*)";
    cin >> algo;
    if (algo == "UCS" || algo == "BFS" || algo == "GBFS" ||
algo == "A*" || algo == "IDA*") {
        break;
    }
    cout << "Pilihan algoritma tidak valid.\n";
}

string heuristic = "H1";
if (algo == "GBFS" || algo == "A*" || algo == "IDA*") {
    while (true) {
        cout << ">> Heuristic apa yang anda pilih?
(H1/H2/H3)";
        cin >> heuristic;
        if (heuristic == "H1" || heuristic == "H2" ||
heuristic == "H3") {
            break;
        }
        cout << "Pilihan heuristic tidak valid.\n";
    }
}

int iterations = 0;
vector<IterationLog> iteration_logs;

auto start_time = chrono::high_resolution_clock::now();
Node result = solve(map, algo, heuristic, iterations,
&iteration_logs);
auto end_time = chrono::high_resolution_clock::now();
const double duration = chrono::duration<double,
std::milli>(end_time - start_time).count();

bool found_solution = !(result.path == "" && (result.state.r
== -1 || result.g == -1));
if (!found_solution) {
    cout << "\nSolusi tidak ditemukan!\n\n";
} else {
    cout << "\nSolusi Yang Ditemukan\n";
    cout << result.path << "\n";
}
```

```
        cout << "Cost dari Solusi: " << result.g << "\n\n";
    }

    vector<State> path_states;
    path_states.push_back({map.start.r, map.start.c, 0});

    if (found_solution) {
        cout << "[Print Initial Map]\n";
        map.printMap(path_states.back());
        cout << "\n";

        State curr = path_states.back();
        for (int i = 0; i < result.path.size(); ++i) {
            char move = result.path[i];
            cout << "Step " << (i + 1) << ": " << move << "\n";
            curr = performSlide(map, curr, move).final_state;
            path_states.push_back(curr);
            cout << "[Print Map at Step " << (i + 1) << "]\n";
            map.printMap(curr);
            cout << "\n";
        }
    }

    cout << ">> Waktu eksekusi: " << fixed << setprecision(5) <<
    duration << " ms\n";
    cout << ">> Banyak iterasi yang dilakukan: " << iterations <<
    " iterasi\n";

    string iteration_filename = "iterasi_pencarian.txt";
    ofstream iter_out(iteration_filename);
    if (iter_out) {
        iter_out << "Algoritma: " << algo << "\n";
        if (algo == "GBFS" || algo == "A*" || algo == "IDA*") {
            iter_out << "Heuristic: " << heuristic << "\n";
        }
        iter_out << "Total iterasi: " << iterations << "\n\n";

        for (const IterationLog& log : iteration_logs) {
            iter_out << "Iterasi " << log.iteration << "\n";
            iter_out << "Posisi: (" << log.state.r << ", " <<
log.state.c << ")\n";
            iter_out << "Mask waypoint: " << log.state.mask <<
"\n";

            iter_out << "Path: " << log.path << "\n";
            iter_out << "g: " << log.g << ", h: " << log.h <<
"\n";

            map.printMap(iter_out, log.state);
            iter_out << "\n";
        }
        iter_out.close();
    }
```

```
        filesystem::path iter_path = filesystem::current_path() /
iteration_filename;
        cout << ">> Konfigurasi state per iterasi disimpan pada "
<< iter_path.string() << "\n";
    } else {
        cout << ">> Gagal menyimpan konfigurasi iterasi.\n";
    }

    string ans;
    if (found_solution) {
        cout << ">> Apakah Anda ingin melakukan playback?
(Ya/Tidak) :";
        cin >> ans;
    } else {
        ans = "Tidak";
    }

    if (ans == "Ya" && found_solution) {
        int step = 0;
        while (true) {
            cout << "\n===== PLAYBACK
=====\\n";
            if (step == 0) {
                cout << "[Print Initial Map]\\n";
                map.printMap(path_states[0]);
            } else {
                cout << "Step " << step << ": " <<
result.path[step - 1] << "\\n";
                cout << "[Print Map at Step " << step << "]\\n";
                map.printMap(path_states[step]);
            }
            cout << "\\nKontrol:\\n";
            cout << "- Arrow kanan: step berikutnya\\n";
            cout << "- Arrow kiri : step sebelumnya\\n";
            cout << "- ESC : lompat ke step tertentu /
keluar playback\\n";

            int key = _getch();
            if (key == 224) {
                int arrow = _getch();
                if (arrow == 77 && step <
static_cast<int>(path_states.size()) - 1) {
                    step++;
                } else if (arrow == 75 && step > 0) {
                    step--;
                }
            } else if (key == 27) {
                cout << "\\nMasukkan step tujuan (0.." <<
(path_states.size() - 1)
```

```
        << ", isi -1 untuk keluar): ";
        int jump_step;
        if (!(cin >> jump_step)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(),
'\n');

            cout << "Input step tidak valid.\n";
            continue;
        }

        if (jump_step == -1) {
            break;
        }

        if (jump_step >= 0 && jump_step <
static_cast<int>(path_states.size())) {
            step = jump_step;
        } else {
            cout << "Step tidak valid.\n";
        }
    }
}

if (found_solution) {
    cout << ">> Apakah Anda ingin menyimpan solusi? (Ya/Tidak)
: ";
    cin >> ans;
} else {
    ans = "Tidak";
}

if (ans == "Ya" && found_solution) {
    string out_filename = "solusi.txt";
    ofstream out(out_filename);
    if (out) {
        out << "Solusi Yang Ditemukan\n";
        out << result.path << "\n";
        out << "Cost dari Solusi: " << result.g << "\n";
        out.close();

        filesystem::path cwd = filesystem::current_path();
        filesystem::path full_path = cwd / out_filename;

        cout << ">> Solusi disimpan pada " <<
full_path.string() << "\n";
    } else {
        cout << ">> Gagal menyimpan solusi.\n";
    }
}
```

```
    return 0;
}
```

File `main.cpp` berfungsi sebagai titik awal interaksi antara pengguna dengan program melalui antarmuka CLI.

- I/O Handling digunakan untuk menerima input berupa path file serta memvalidasi pilihan algoritma dan heuristik dari pengguna.
- Kalkulasi Metrik dilakukan menggunakan `<chrono>` untuk mengukur waktu eksekusi program secara real-time dalam satuan milidetik.
- Playback Interaktif memungkinkan pengguna melihat kembali langkah-langkah solusi melalui terminal. Fitur ini memanfaatkan `<conio.h>` sehingga pengguna dapat menavigasi langkah menggunakan tombol panah (Arrow Keys) atau berpindah ke iterasi tertentu dengan tombol ESC.
- Penyimpanan Output dilakukan menggunakan pustaka `<filesystem>` dan `<fstream>`. Seluruh proses iterasi pencarian disimpan pada file `iterasi_pencarian.txt`, sedangkan solusi akhir disimpan ke dalam `solusi.txt`.

3.2.5 GUI (MainWindow.hpp dan MainWindow.cpp)

```
void MainWindow::run() {
    SetConfigFlags(FLAG_WINDOW_RESIZABLE | FLAG_MSAA_4X_HINT);
    InitWindow(kInitialWidth, kInitialHeight, "Ice Sliding Puzzle
Solver - Raylib GUI");
    try {
        exe_dir_ =
std::filesystem::absolute(std::filesystem::path(GetApplicationDire
ctory()));
    } catch (...) {
        exe_dir_ = std::filesystem::current_path();
    }
    const char* segoe_path = "C:/Windows/Fonts/segoeui.ttf";
    const char* arial_path = "C:/Windows/Fonts/arial.ttf";
    if (FileExists(segoe_path)) {
        ui_font_ = LoadFontEx(segoe_path, 32, nullptr, 0);
        font_loaded_ = true;
    } else if (FileExists(arial_path)) {
        ui_font_ = LoadFontEx(arial_path, 32, nullptr, 0);
        font_loaded_ = true;
    }
    if (font_loaded_) {
        SetTextureFilter(ui_font_.texture,
TEXTURE_FILTER_BILINEAR);
    }
}
```

```
board_widget_.setFont(&ui_font_, font_loaded_);
refreshTxtCandidates();

MaximizeWindow();
window_maximized_ = true;
SetTargetFPS(60);

while (!WindowShouldClose()) {
    handleInput();
    updatePlaybackFromKeyboard();
    updateAutoplay();

    BeginDrawing();
    draw();
    EndDrawing();
}

if (font_loaded_) {
    UnloadFont(ui_font_);
}
CloseWindow();
}
```

Sebagai bagian dari fitur tambahan, program ini dilengkapi dengan antarmuka grafis interaktif yang dikembangkan menggunakan library Raylib. Modul MainWindow berperan sebagai pengendali utama aplikasi, mulai dari pengelolaan window loop, penanganan event, hingga pengaturan tata letak antarmuka. Tampilan GUI dibagi menjadi empat panel utama dengan fungsi yang berbeda.

3.2.6 Modul Render Papan (BoardWidget.hpp dan BoardWidget.cpp)

```
void BoardWidget::draw(const Map* map, const State* state, const
Rectangle& area) const {
    DrawRectangleRec(area, kGridBg);

    if (map == nullptr || state == nullptr) {
        if (font_loaded_ && font_ != nullptr) {
            DrawTextEx(*font_, "Load map lalu tekan SOLVE",
{area.x + 20, area.y + area.height / 2.0f}, 24.0f, 1.0f,
kTextLight);
        } else {
            DrawText("Load map lalu tekan SOLVE",
static_cast<int>(area.x + 20), static_cast<int>(area.y +
area.height / 2.0f), 24, kTextLight);
        }
    }
    return;
}
```

```
}

const float margin = 20.0f;
const float usable_w = area.width - 2.0f * margin;
const float usable_h = area.height - 2.0f * margin;
const float cell = (map->cols == 0 || map->rows == 0)
    ? 1.0f
    : std::min(usable_w /
static_cast<float>(map->cols), usable_h /
static_cast<float>(map->rows));

const float board_w = cell * static_cast<float>(map->cols);
const float board_h = cell * static_cast<float>(map->rows);
const float ox = area.x + (area.width - board_w) * 0.5f;
const float oy = area.y + (area.height - board_h) * 0.5f;

for (int r = 0; r < map->rows; ++r) {
    for (int c = 0; c < map->cols; ++c) {
        char ch = map->grid[r][c];
        if (ch >= '0' && ch <= '9' && (ch - '0') <
state->mask) {
            ch = '*';
        }

        Rectangle tile{ox + c * cell, oy + r * cell, cell,
cell};
        bool is_outer_ring = (r == 0 || r == map->rows - 1 ||
c == 0 || c == map->cols - 1);
        Color fill = tileColor(ch);

        // For border walls, use dark tiles and represent the
boundary
        // as one thick white outline around the board.
        if (ch == 'X' && is_outer_ring) {
            fill = kTileNormal;
        }

        DrawRectangleRec(tile, fill);
        DrawRectangleLinesEx(tile, 1.0f, kTileGrid);

        if (ch >= '0' && ch <= '9') {
            char txt[2] = {ch, '\\0'};
            const int font_size = static_cast<int>(cell *
0.33f);
            int tw = 0;
            if (font_loaded_ && font_ != nullptr) {
                tw = static_cast<int>(MeasureTextEx(*font_,
txt, static_cast<float>(font_size), 1.0f).x);
                DrawTextEx(*font_, txt, {tile.x + (tile.width
- tw) * 0.5f, tile.y + tile.height * 0.22f},
```



```
static_cast<float>(font_size), 1.0f, BLACK);
    } else {
        tw = MeasureText(txt, font_size);
        DrawText(txt, static_cast<int>(tile.x +
(tile.width - tw) * 0.5f), static_cast<int>(tile.y + tile.height *
0.22f), font_size, BLACK);
    }
}
}

Rectangle actor{
    ox + state->c * cell + 4.0f,
    oy + state->r * cell + 4.0f,
    cell - 8.0f,
    cell - 8.0f
};
DrawRectangleRec(actor, kActor);
DrawRectangleLinesEx(actor, 1.0f, kActorOutline);
const int fs = static_cast<int>(cell * 0.30f);
if (font_loaded_ && font_ != nullptr) {
    const int zw = static_cast<int>(MeasureTextEx(*font_, "Z",
static_cast<float>(fs), 1.0f).x);
    DrawTextEx(*font_, "Z", {actor.x + (actor.width - zw) *
0.5f, actor.y + actor.height * 0.22f}, static_cast<float>(fs),
1.0f, WHITE);
} else {
    const int zw = MeasureText("Z", fs);
    DrawText("Z", static_cast<int>(actor.x + (actor.width -
zw) * 0.5f), static_cast<int>(actor.y + actor.height * 0.22f), fs,
WHITE);
}

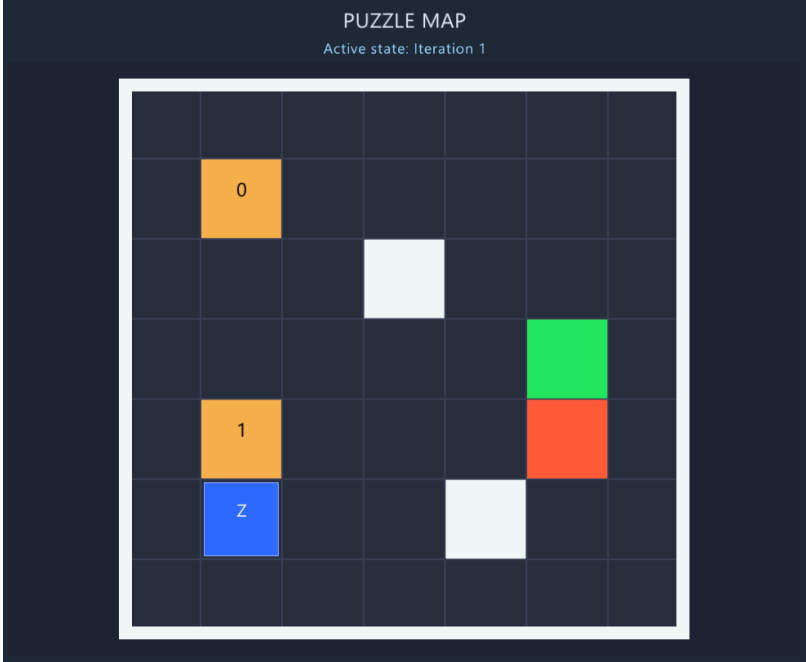
Rectangle board_rect{ox, oy, board_w, board_h};
float border_thickness = std::max(4.0f, cell * 0.16f);
DrawRectangleLinesEx(board_rect, border_thickness, kTileWall);
}
```

Modul ini secara khusus menangani proses rendering visual papan Ice Sliding Puzzle pada layar GUI.

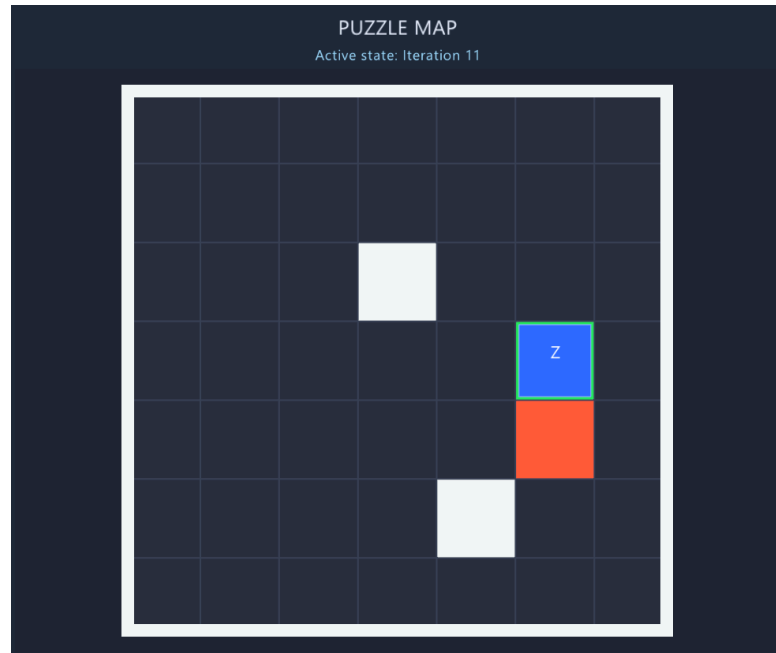
BAB 4

PENGUJIAN DAN ANALISIS HASIL

5.1 Test Case 1

Input
7 7 XXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999
PUZZLE MAP Active state: Iteration 1 

Output



```
>> Masukan file input :
    test/input/input1.txt
>> Algoritma apa yang anda pilih? (UCS/BFS/GBFS/A*/IDA*)
    GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
    H2
```

===== KONFIGURASI / ITERASI PENCARIAN =====

Total iterasi: 11

Iterasi 1

Posisi: (5, 1)

Mask waypoint: 0

Path:

g: 0, h: 4

XXXXXXXX

X0****X

X**X**X

X****OX

X1***LX

XZ**X*X

XXXXXXXX

Iterasi 2

Posisi: (5, 3)
Mask waypoint: 0
Path: R
g: 7, h: 4.47214
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Iterasi 3
Posisi: (3, 3)
Mask waypoint: 0
Path: RU
g: 17, h: 2.82843
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Iterasi 4
Posisi: (3, 1)
Mask waypoint: 0
Path: RUL
g: 27, h: 2
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Iterasi 5
Posisi: (1, 1)
Mask waypoint: 1
Path: RULU
g: 37, h: 3
XXXXXXX
XZ****X
X**X**X
X****OX

X1***LX
X***X*X
XXXXXXX

Iterasi 6
Posisi: (3, 5)
Mask waypoint: 0
Path: RUR
g: 26, h: 4.47214
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

Iterasi 7
Posisi: (1, 5)
Mask waypoint: 0
Path: RURU
g: 36, h: 4
XXXXXXX
X0***ZX
X**X**X
X****OX
X1***LX
X***X*X
XXXXXXX

Iterasi 8
Posisi: (5, 1)
Mask waypoint: 2
Path: RULUD
g: 61, h: 4.47214
XXXXXXX
X*****X
X**X**X
X****OX
X****LX
XZ**X*X
XXXXXXX

Iterasi 9
Posisi: (5, 3)
Mask waypoint: 2
Path: RULUDR

g: 68, h: 2.82843

XXXXXXX

X*****X

X**X**X

X****OX

X****LX

X**ZX**X

XXXXXXX

Iterasi 10

Posisi: (3, 3)

Mask waypoint: 2

Path: RULUDRU

g: 78, h: 2

XXXXXXX

X*****X

X**X**X

X**Z*OX

X****LX

X***X*X

XXXXXXX

Iterasi 11

Posisi: (3, 5)

Mask waypoint: 2

Path: RULUDRUR

g: 87, h: 0

XXXXXXX

X*****X

X**X**X

X****ZX

X****LX

X***X*X

XXXXXXX

Solusi Yang Ditemukan : RULUDRUR

Cost dari Solusi : 87

>> Waktu eksekusi: 0.37140 ms

>> Banyak iterasi yang dilakukan: 11 iterasi

>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :

Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :

Ya

5.2 Test Case 2

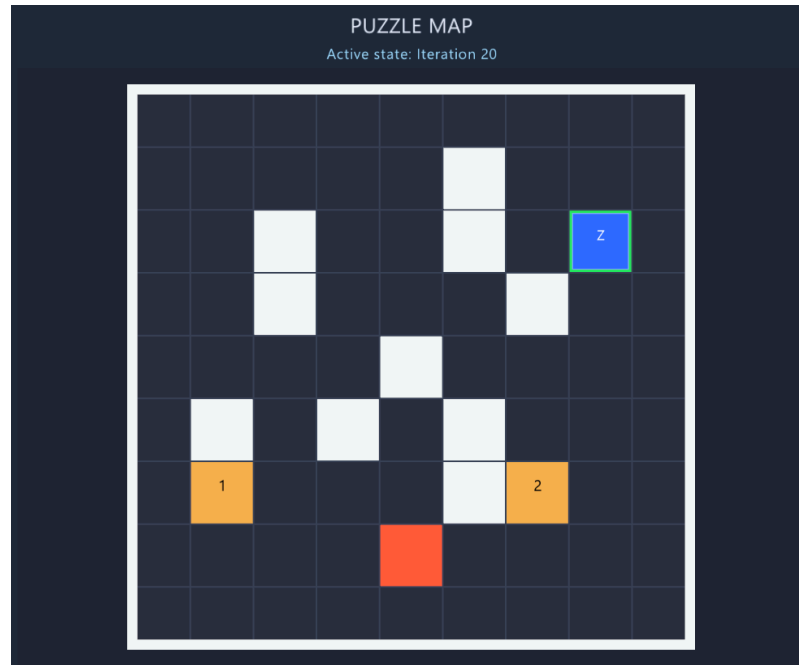
Input

```

9 9
XXXXXXXXXX
XZ***X**X
X*X*0X*OX
X*X***X*X
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX
111111111
111111111
111111111
111111111
111111111
111111111
111111111
111111111
111111111
111111111
111111111
111111111

```

Output



```
>> Masukan file input :
    test/input/input2.txt
>> Algoritma apa yang anda pilih? (UCS/BFS/GBFS/A*/IDA*)
    GBFS
>> Heuristic apa yang anda pilih? (H1/H2/H3)
    H2
```

===== KONFIGURASI / ITERASI PENCARIAN =====

Total iterasi: 20

Iterasi 1

Posisi: (1, 1)

Mask waypoint: 0

Path:

g: 0, h: 3.16228

XXXXXXXXXX

XZ***X**X

X*X*0X*OX

X*X***X*X

X***X***X

XX*X*X**X

X1***X2*X

X***L***X

XXXXXXXXXX

Iterasi 2
Posisi: (1, 4)
Mask waypoint: 0
Path: R
g: 3, h: 1
XXXXXXXXXX
X***ZX**X
X*X*0X*OX
X*X***X*X
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX

Iterasi 3
Posisi: (4, 1)
Mask waypoint: 0
Path: D
g: 3, h: 3.60555
XXXXXXXXXX
X***X**X
X*X*0X*OX
X*X***X*X
XZ**X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX

Iterasi 4
Posisi: (4, 3)
Mask waypoint: 0
Path: DR
g: 5, h: 2.23607
XXXXXXXXXX
X***X**X
X*X*0X*OX
X*X***X*X
X**ZX***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX

Iterasi 5

Posisi: (1, 3)
Mask waypoint: 0
Path: DRU
g: 8, h: 1.41421
XXXXXXXXXX
X**Z*X**X
X*X*0X*OX
X*X***X*X
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX

Iterasi 6
Posisi: (3, 4)
Mask waypoint: 1
Path: RD
g: 5, h: 4.24264
XXXXXXXXXX
X****X**X
X*X**X*OX
X*X*Z*X*X
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX

Iterasi 7
Posisi: (3, 3)
Mask waypoint: 1
Path: RDL
g: 6, h: 3.60555
XXXXXXXXXX
X****X**X
X*X**X*OX
X*XZ**X*X
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX

Iterasi 8
Posisi: (4, 3)
Mask waypoint: 1

Path: RDLD
g: 7, h: 2.82843
XXXXXXXXXX
X***X**X
X*X**X*OX
X*X***X*X
X**ZX***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX

Iterasi 9
Posisi: (4, 1)
Mask waypoint: 1
Path: RDLDL
g: 9, h: 2
XXXXXXXXXX
X***X**X
X*X**X*OX
X*X***X*X
XZ**X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX

Iterasi 10
Posisi: (3, 5)
Mask waypoint: 1
Path: RDR
g: 6, h: 5
XXXXXXXXXX
X***X**X
X*X**X*OX
X*X**ZX*X
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX

Iterasi 11
Posisi: (4, 5)
Mask waypoint: 1
Path: RDRD
g: 7, h: 4.47214

```
XXXXXXXXXX
X***X**X
X*X**X*OX
X*X***X*X
X***XZ**X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX
```

Iterasi 12

Posisi: (1, 1)

Mask waypoint: 1

Path: RDLDLU

g: 12, h: 5

```
XXXXXXXXXX
```

```
XZ***X**X
```

```
X*X**X*OX
```

```
X*X***X*X
```

```
X***X***X
```

```
XX*X*X**X
```

```
X1***X2*X
```

```
X***L***X
```

```
XXXXXXXXXX
```

Iterasi 13

Posisi: (1, 3)

Mask waypoint: 1

Path: RDLU

g: 8, h: 5.38516

```
XXXXXXXXXX
```

```
X**Z*X**X
```

```
X*X**X*OX
```

```
X*X***X*X
```

```
X***X***X
```

```
XX*X*X**X
```

```
X1***X2*X
```

```
X***L***X
```

```
XXXXXXXXXX
```

Iterasi 14

Posisi: (1, 4)

Mask waypoint: 1

Path: RDLDLUR

g: 15, h: 5.83095

```
XXXXXXXXXX
```

```
X***ZX**X
```

```
X**X**OX
X**X**X
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX
```

```
Iterasi 15
Posisi: (4, 7)
Mask waypoint: 1
Path: RDRDR
g: 9, h: 6.32456
XXXXXXXXXX
X****X**X
X**X**OX
X**X**X
X***X**ZX
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX
```

```
Iterasi 16
Posisi: (7, 7)
Mask waypoint: 1
Path: RDRDRD
g: 12, h: 6.08276
XXXXXXXXXX
X****X**X
X**X**OX
X**X**X
X***X***X
XX*X*X**X
X1***X2*X
X***L**ZX
XXXXXXXXXX
```

```
Iterasi 17
Posisi: (1, 7)
Mask waypoint: 1
Path: RDRDRU
g: 12, h: 7.81025
XXXXXXXXXX
X****X*ZX
X**X**OX
X**X**X
```

```
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX
```

```
Iterasi 18
Posisi: (1, 6)
Mask waypoint: 1
Path: RDRDRUL
g: 13, h: 7.07107
XXXXXXXXXX
X****XZ*X
X*X**X*OX
X*X***X*X
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX
```

```
Iterasi 19
Posisi: (2, 6)
Mask waypoint: 1
Path: RDRDRULD
g: 14, h: 6.40312
XXXXXXXXXX
X****X**X
X*X**XZOX
X*X***X*X
X***X***X
XX*X*X**X
X1***X2*X
X***L***X
XXXXXXXXXX
```

```
Iterasi 20
Posisi: (2, 7)
Mask waypoint: 1
Path: RDRDRULDR
g: 15, h: 7.2111
XXXXXXXXXX
X****X**X
X*X**X*ZX
X*X***X*X
X***X***X
XX*X*X**X
```

```
X1***X2*X
X***L***X
XXXXXXXXXX
```

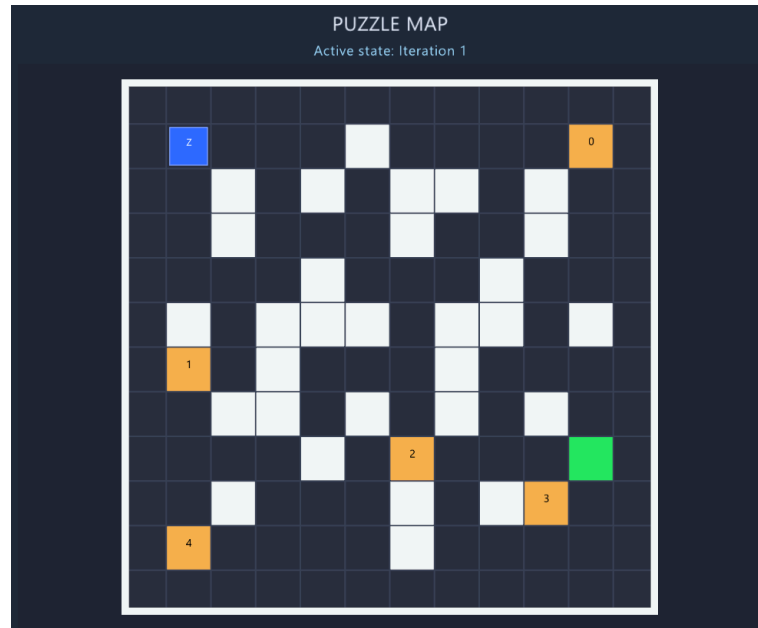
Solusi tidak ditemukan!

>> Waktu eksekusi: 0.04620 ms

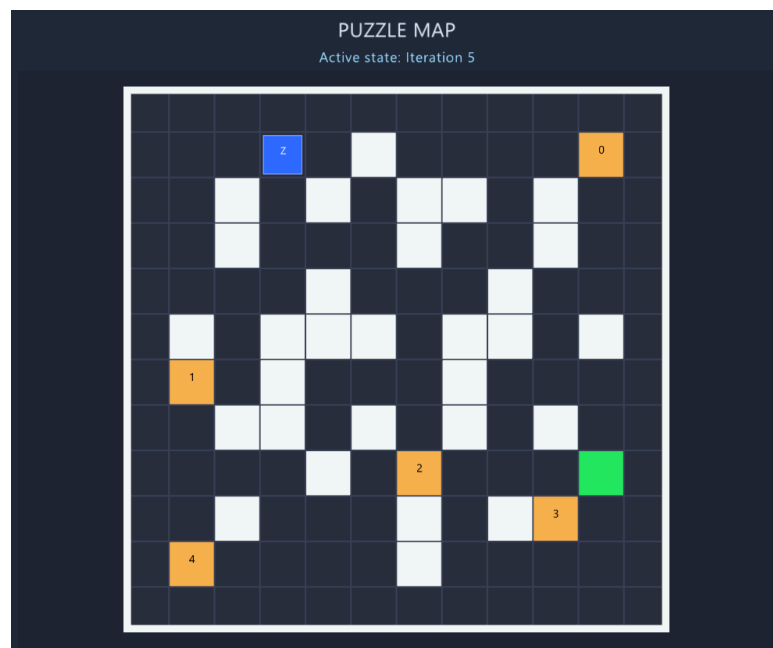
>> Banyak iterasi yang dilakukan: 20 iterasi

5.3 Test Case 3

Input
<pre>12 12 XXXXXXXXXXXXXXXX XZ***X***0X X*X*X*XX*X*X X*X***X**X*X X***X***X**X XX*XXX*XX*XX X1*X***X***X X*XX*X*X*X*X X***X*2***OX X*X***X*X3*X X4***X***X XXXXXXXXXXXXXXXX 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111 111111111111</pre>



Output



>> Masukan file input :

test/input/input3.txt

>> Algoritma apa yang anda pilih? (UCS/BFS/GBFS/A*/IDA*)

GBFS

>> Heuristic apa yang anda pilih? (H1/H2/H3)

H2

===== KONFIGURASI / ITERASI PENCARIAN =====

Total iterasi: 5

Iterasi 1

Posisi: (1, 1)

Mask waypoint: 0

Path:

g: 0, h: 9

XXXXXXXXXXXXX

XZ***X***0X

X*X*X*XX*X*X

X*X***X**X*X

X***X***X**X

XX*XXX*XX*XX

X1*X***X***X

X*XX*X*X*X*X

X***X*2***OX

X*X***X*X3*X

X4***X***X

XXXXXXXXXXXXX

Iterasi 2

Posisi: (1, 4)

Mask waypoint: 0

Path: R

g: 3, h: 6

XXXXXXXXXXXXX

X***ZX***0X

X*X*X*XX*X*X

X*X***X**X*X

X***X***X**X

XX*XXX*XX*XX

X1*X***X***X

X*XX*X*X*X*X

X***X*2***OX

X*X***X*X3*X

X4***X***X

XXXXXXXXXXXXX

Iterasi 3

Posisi: (4, 1)

Mask waypoint: 0

Path: D

g: 3, h: 9.48683

XXXXXXXXXXXXX

```
X****X****0X
X*X*X*XX*X*X
X*X***X**X*X
XZ**X***X**X
XX*XXX*XX*XX
X1*X***X***X
X*XX*X*X*X*X
X***X*2***OX
X*X***X*X3*X
X4****X****X
XXXXXXXXXXXXXX
```

Iterasi 4

Posisi: (4, 3)

Mask waypoint: 0

Path: DR

g: 5, h: 7.61577

```
XXXXXXXXXXXXXX
```

```
X****X****0X
X*X*X*XX*X*X
X*X***X**X*X
X**ZX***X**X
XX*XXX*XX*XX
X1*X***X***X
X*XX*X*X*X*X
X***X*2***OX
X*X***X*X3*X
X4****X****X
XXXXXXXXXXXXXX
```

Iterasi 5

Posisi: (1, 3)

Mask waypoint: 0

Path: DRU

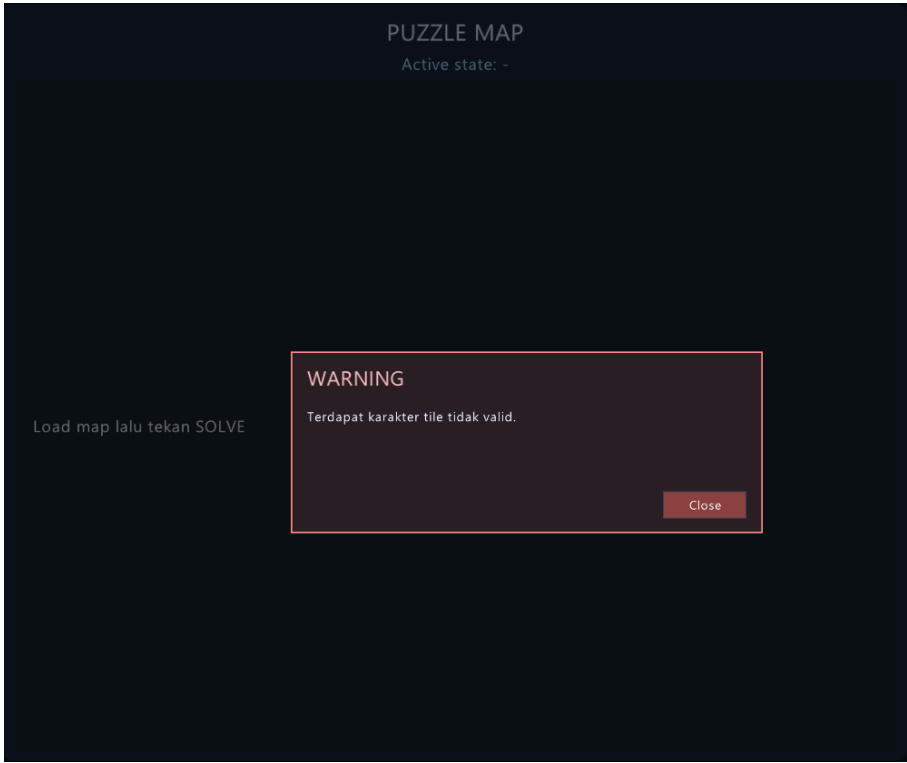
g: 8, h: 7

```
XXXXXXXXXXXXXX
```

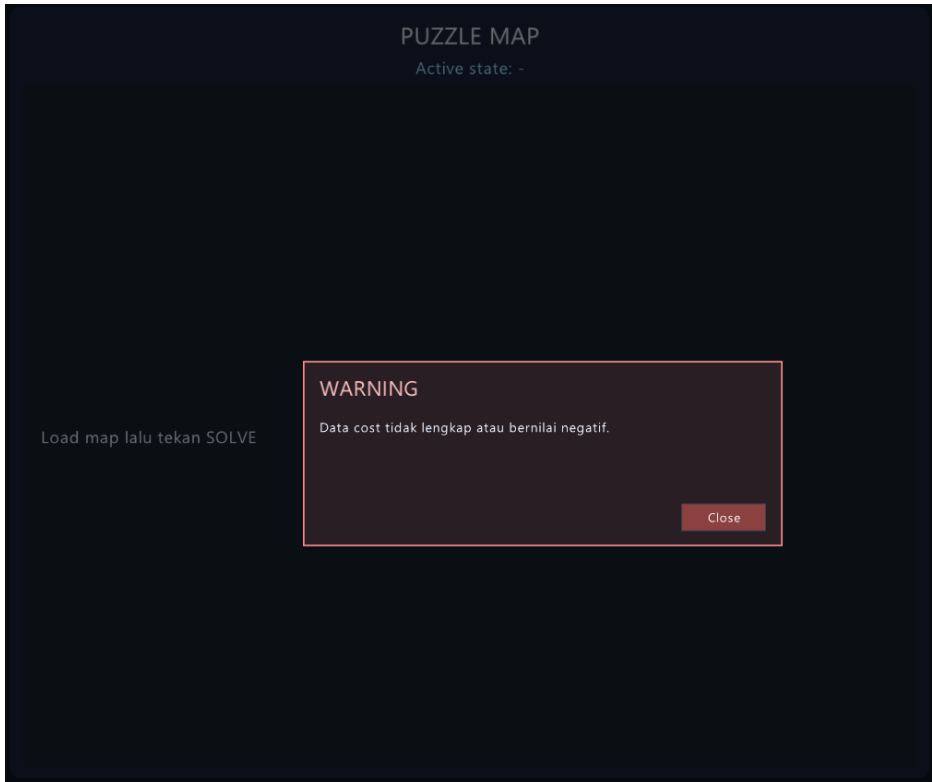
```
X**ZX****0X
X*X*X*XX*X*X
X*X***X**X*X
X***X***X**X
XX*XXX*XX*XX
X1*X***X***X
X*XX*X*X*X*X
X***X*2***OX
X*X***X*X3*X
X4****X****X
XXXXXXXXXXXXXX
```

Solusi tidak ditemukan!
>> Waktu eksekusi: 0.07170 ms
>> Banyak iterasi yang dilakukan: 5 iterasi

5.4 Test Case 4

Input
5 6 XXXXXX XZ@0OX X****X X****X XXXXXX 1
Output


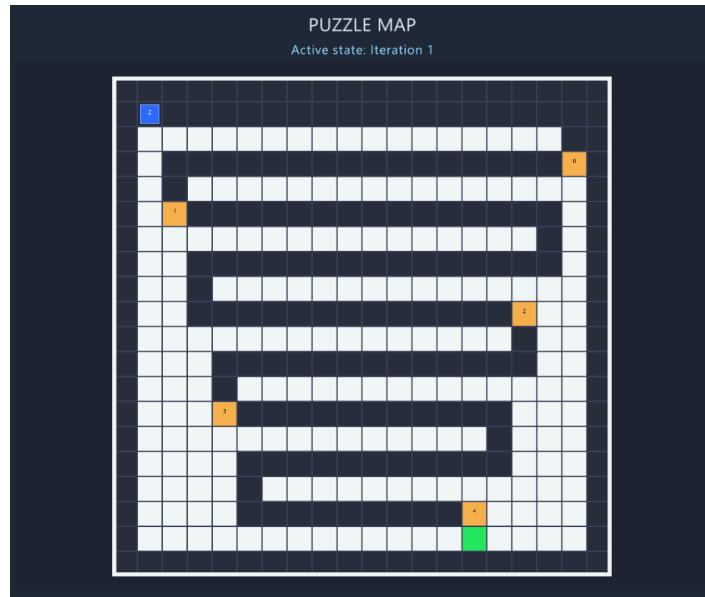
5.5 Test Case 5

Input
5 5 XXXXX XZ00X X***X X***X XXXXX 11111 11-311 11111 11111 11111
Output
 The screenshot shows a dark-themed application window titled "PUZZLE MAP". Below the title, it says "Active state: -". In the center, there is a warning dialog box with a red border. The dialog box has the title "WARNING" and the message "Data cost tidak lengkap atau bernilai negatif." (Data cost is incomplete or has a negative value). At the bottom right of the dialog box is a "Close" button. To the left of the dialog box, the text "Load map lalu tekan SOLVE" is visible.

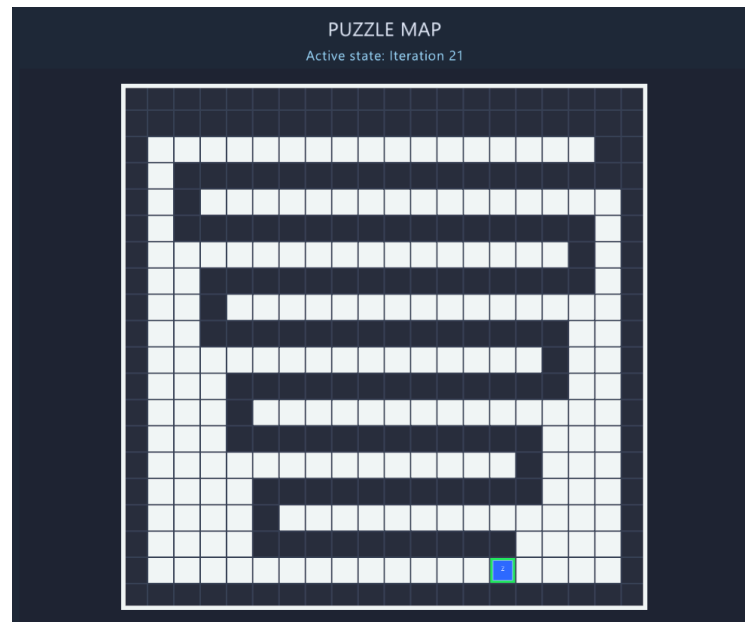
5.6 Test Case 6

Input

[illegible]



Output



```
>> Masukan file input :  
    test/input/input11_valid_large_many_iterations.txt  
>> Algoritma apa yang anda pilih? (UCS/BFS/GBFS/A*/IDA*)  
    GBFS  
>> Heuristic apa yang anda pilih? (H1/H2/H3)  
    H2
```

===== KONFIGURASI / ITERASI PENCARIAN =====

Total iterasi: 21

Iterasi 1

Posisi: (1, 1)

Mask waypoint: 0

Path:

g: 0, h: 17.1172

XXXXXXXXXXXXXXXXXXXXX

XZXXXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXXXX*X

XXXXXXXXXXXXXXXXXX0X

XX*XXXXXXXXXXXXXXXXXXXXX

XX1XXXXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXXXX*XX

XXXXXXXXXXXXXXXXXXXXX

XXX*XXXXXXXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXX2XXX

XXXXXXXXXXXXXXXXXXXXX*XXX

XXXXXXXXXXXXXXXXXXXXXX

XXXX*XXXXXXXXXXXXXXXXXXXXX

XXXX3XXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXXXX*XXXX

XXXXX*XXXXXXXXXXXXXXXXXXXXX

XXXXX*XXXXXXXXXXXXXXXXXXXXX

XXXXX*XXXXXXXXXXXXXXXXXXXXX

XXXXX*XXXXXXXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXXXXOXXXXX

XXXXXXXXXXXXXXXXXXXXX

Iterasi 2

Posisi: (1, 18)

Mask waypoint: 0

Path: R

g: 17, h: 2

XXXXXXXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXXXX*X

XXXXXXXXXXXXXXXXXX0X

XX*XXXXXXXXXXXXXXXXXXXXX

XX1XXXXXXXXXXXXXXXXXX

XXXXXXXXXXXXXXXXXXXXX*XX

XXXXXXXXXXXXXXXXXXXXX

```
XXX*XXXXXXXXXXXXXXXXXX
XXX*****2XXX
XXXXXXXXXXXXXXXXXX*XXX
XXX*****XXX
XXX*XXXXXXXXXXXXXXXXXX
XXX3*****XXX
XXXXXXXXXXXXXXXXXX*XXX
XXX*****XXX
XXX*XXXXXXXXXXXXXXXXXX
XXX*****4XXXX
XXXXXXXXXXXXXXXXXXOXXXX
XXXXXXXXXXXXXXXXXXXXXX
```

Iterasi 3

Posisi: (3, 18)

Mask waypoint: 1

Path: RD

g: 19, h: 16.1245

```
XXXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXX*X
XX*****ZX
XX*XXXXXXXXXXXXXXXXXX
XX1*****XX
XXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXXXX
XXX*****2XXX
XXXXXXXXXXXXXXXXXX*XXX
XXX*****XXX
XXX*XXXXXXXXXXXXXXXXXX
XXX3*****XXX
XXXXXXXXXXXXXXXXXX*XXX
XXX*****XXX
XXX*XXXXXXXXXXXXXXXXXX
XXX*****4XXXX
XXXXXXXXXXXXXXXXXXOXXXX
XXXXXXXXXXXXXXXXXXXXXX
```

Iterasi 4

Posisi: (3, 2)

Mask waypoint: 1

Path: RDL

g: 35, h: 2

```
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX*X
XXZ*****X
XX*XXXXXXXXXXXXXXXXX
XXI*****XX
XXXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXXX
XXX*****2XXX
XXXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXXX
XXXX3*****XXXX
XXXXXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXXX
```

Iterasi 5

Posisi: (5, 2)

Mask waypoint: 2

Path: RDLD

g: 37, h: 14.5602

```
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXXX
XXZ*****XX
XXXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXXX
XXX*****2XXX
XXXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXXX
XXXX3*****XXXX
XXXXXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
```

```
XXXXX*XXXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXXXXXXX
```

Iterasi 6

Posisi: (5, 17)

Mask waypoint: 2

Path: RDLDR

g: 52, h: 4.12311

```
XXXXXXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXXXXXXX
XX*****ZXX
XXXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXXXXXXX
XXX*****2XXX
XXXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXXXXXXX
XXXX3*****XXXX
XXXXXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXXXXXXX
```

Iterasi 7

Posisi: (7, 17)

Mask waypoint: 2

Path: RDLDRD

g: 54, h: 2.23607

```
XXXXXXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXXX*XX
```

```
XXX*****ZXX
XXX*XXXXXXXXXXXXXXXXX
XXX*****2XXX
XXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXXX
XXXX3*****XXXX
XXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXXXX
```

Iterasi 8

Posisi: (7, 3)

Mask waypoint: 2

Path: RDLDRDL

g: 68, h: 13.1529

```
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXX*XX
XXXZ*****XX
XXX*XXXXXXXXXXXXXXXXX
XXX*****2XXX
XXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXXX
XXXX3*****XXXX
XXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXXX
```

Iterasi 9

Posisi: (9, 3)

Mask waypoint: 2

Path: RDLDRDLD

g: 70, h: 13

```
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX
XX*****X
XX*XXXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXXX
XXX*****XX
XXX*XXXXXXXXXXXXXXXX
XXXZ*****2XXX
XXXXXXXXXXXXXXXXXXXXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXX
XXXX3*****XXXX
XXXXXXXXXXXXXXXXXXXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXX
```

Iterasi 10

Posisi: (9, 16)

Mask waypoint: 3

Path: RDLDRDLDR

g: 83, h: 12.6491

```
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX
XX*****X
XX*XXXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXXX
XXX*****XX
XXX*XXXXXXXXXXXXXXXX
XXX*****ZXXX
XXXXXXXXXXXXXXXXXXXXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXX
XXXX3*****XXXX
XXXXXXXXXXXXXXXXXXXXX
```

```
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXX
```

Iterasi 11

Posisi: (9, 3)

Mask waypoint: 3

Path: RDLDRDLRL

g: 96, h: 4.12311

```
XXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXX
XXXZ*****XXX
XXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXX
XXXX3*****XXXX
XXXXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXX
```

Iterasi 12

Posisi: (7, 3)

Mask waypoint: 3

Path: RDLDRDLRLU

g: 98, h: 6.08276

```
XXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXX
XX*****XX
```

```
XXXXXXXXXXXXXXXXXXXX*XX
XXXZ*****XX
XXX*XXXXXXXXXXXXXXXX
XXX*****XXX
XXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXX
XXXX3*****XXX
XXXXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXX
```

Iterasi 13

Posisi: (11, 16)

Mask waypoint: 3

Path: RDLDRDLDRD

g: 85, h: 12.1655

```
XXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXX
XXX*****XXX
XXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****ZXXX
XXXX*XXXXXXXXXXXXXXXX
XXXX3*****XXX
XXXXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXX
```

Iterasi 14

Posisi: (11, 4)

Mask waypoint: 3
Path: RDLDRDLDRDL
g: 97, h: 2
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXXX
XXX*****XXX
XXXXXXXXXXXXXXXXXXXXX*XXX
XXXXZ*****XXX
XXXX*XXXXXXXXXXXXXXXXX
XXXX3*****XXXX
XXXXXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXXX

Iterasi 15
Posisi: (13, 4)
Mask waypoint: 4
Path: RDLDRDLDRDL
g: 99, h: 10.7703
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXXX
XXX*****XXX
XXXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXXX
XXXXZ*****XXXX

```
XXXXXXXXXXXXXXXXXXXX*XXXXX
XXXXXX*XXXXXXXXXXXXX
XXXXXX*XXXXXXXXXXXXXXXXXXXX
XXXXXX*XXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXX
```

Iterasi 16

Posisi: (13, 15)

Mask waypoint: 4

Path: RDLDRDLDRDLDR

g: 110, h: 4.12311

```
XXXXXXXXXXXXXXXXXXXXX
X*XXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXX*X
XX*XXXXXXXXXXXXX
XX*XXXXXXXXXXXXXXXXXXXX
XX*XXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXX*XX
XXX*XXXXXXXXXXXXX
XXX*XXXXXXXXXXXXXXXXXXXX
XXX*XXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXX*XXX
XXXX*XXXXXXXXXXXXX
XXXX*XXXXXXXXXXXXX
XXXX*XXXXXXXXXXXXXXXXXXXX
XXXX*XXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXX*XXXX
XXXX*XXXXXXXXXXXXX
XXXX*XXXXXXXXXXXXXXXXXXXX
XXXX*XXXXXXXXXXXXX
XXXX*XXXXXXXXXXXXXXXXXXXX
XXXX*XXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXX
```

Iterasi 17

Posisi: (15, 15)

Mask waypoint: 4

Path: RDLDRDLDRDLDRD

g: 112, h: 2.23607

```
XXXXXXXXXXXXXXXXXXXXX
X*XXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXX*X
XX*XXXXXXXXXXXXX
XX*XXXXXXXXXXXXXXXXXXXX
```



```
XX*****XX
XXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXX
XXX*****XXX
XXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXX
XXXX*****XXXX
XXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****ZXXXX
XXXXX*XXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXX
```

Iterasi 18

Posisi: (15, 5)

Mask waypoint: 4

Path: RDLDRDLDRDLDRDL

g: 122, h: 9.21954

```
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXX
XXX*****XXX
XXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXX
XXXX*****XXXX
XXXXXXXXXXXXXXXXXXXX*XXXX
XXXXXZ*****XXXX
XXXXX*XXXXXXXXXXXXXXXX
XXXXX*****4XXXXX
XXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXX
```

Iterasi 19

Posisi: (17, 5)
Mask waypoint: 4
Path: RDLDRDLDRDLDRDL
g: 124, h: 9
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXXX
XXX*****XXX
XXXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXXX
XXXX*****XXXX
XXXXXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXXX
XXXXXXZ*****4XXXXX
XXXXXXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXXX

Iterasi 20
Posisi: (17, 14)
Mask waypoint: 5
Path: RDLDRDLDRDLDRDLDR
g: 133, h: 1
XXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXXX
XXX*****XXX
XXXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXXX

```
XXXX*****XXXX
XXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXX
XXXXX*****ZXXXXX
XXXXXXXXXXXXXXXXXOXXXXX
XXXXXXXXXXXXXXXXXXXXXXX
```

Iterasi 21

Posisi: (18, 14)

Mask waypoint: 5

Path: RDLDRDLDRDLDRDLDRD

g: 134, h: 0

```
XXXXXXXXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXXXXXX*X
XX*****X
XX*XXXXXXXXXXXXXXXXXXXX
XX*****XX
XXXXXXXXXXXXXXXXXXXX*XX
XXX*****XX
XXX*XXXXXXXXXXXXXXXXXXXX
XXX*****XXX
XXXXXXXXXXXXXXXXXXXX*XXX
XXXX*****XXX
XXXX*XXXXXXXXXXXXXXXXXXXX
XXXX*****XXXX
XXXXXXXXXXXXXXXXXXXX*XXXX
XXXXX*****XXXX
XXXXX*XXXXXXXXXXXXXXXXXXXX
XXXXX*****XXXXX
XXXXXXXXXXXXXXXXXXXXZXXXXX
XXXXXXXXXXXXXXXXXXXXXXX
```

Solusi Yang Ditemukan : RDLDRDLDRDLDRDLDRD

Cost dari Solusi : 134

>> Waktu eksekusi: 0.31560 ms

>> Banyak iterasi yang dilakukan: 21 iterasi

>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :

Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :

Ya

LAMPIRAN

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

PERNYATAAN TIDAK MELAKUKAN KECURANGAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Dika Pramudya Nugraha

Daniel Putra Rywandi S

DAFTAR PUSTAKA

- [1] [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2023-2024/Algoritma-Divide-and-Conquer-\(2024\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2023-2024/Algoritma-Divide-and-Conquer-(2024)-Bagian1.pdf)
- [2] GfG. (2024, February 22). Divide and conquer. GeeksforGeeks.
<https://www.geeksforgeeks.org/divide-and-conquer/#what-is-divide-and-conquer>
- [3] *Divide and conquer algorithm*. (n.d.). <https://www.programiz.com/dsa/divide-and-conquer>
- [4] Samet, H. (1990). *The Design and Analysis of Spatial Data Structures*. Addison-Wesley.